Infrastructure as Code (IaC) Intro

Arseniy Buzyan

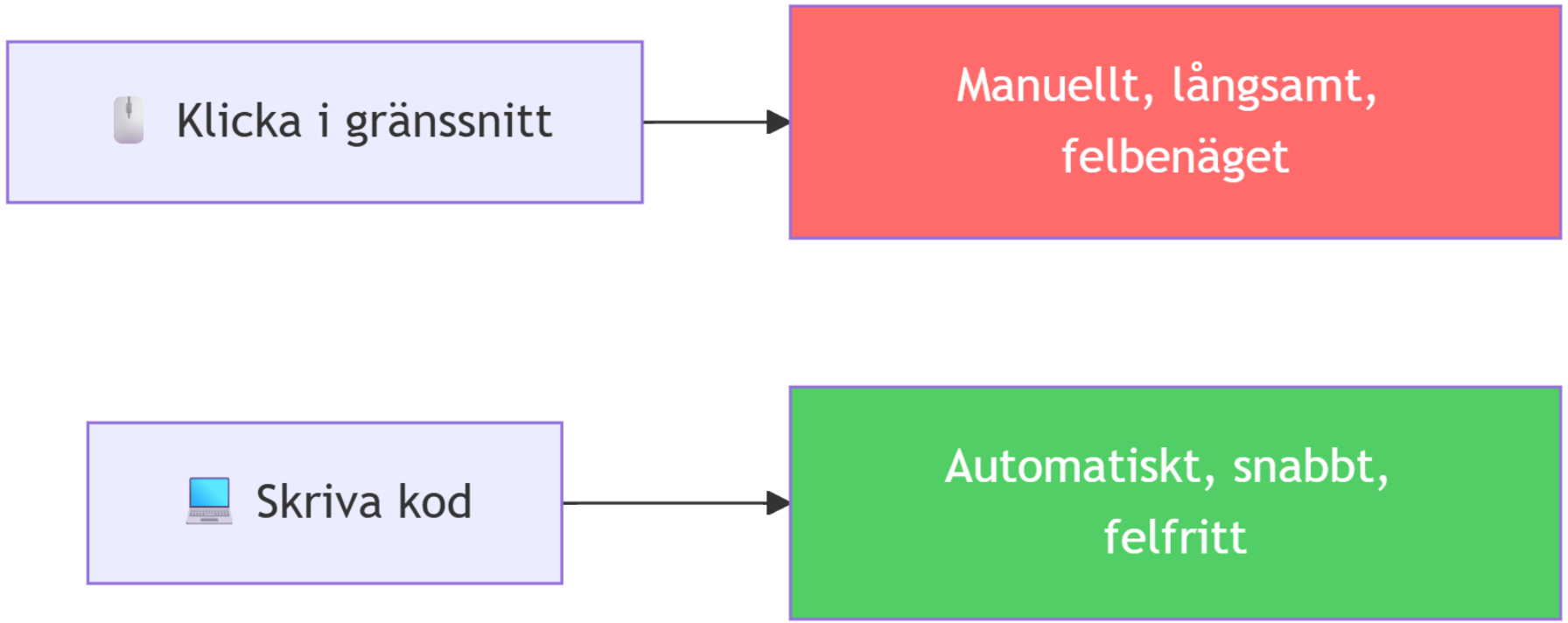
Varför Infrastructure as Code?

Vi har redan lärt oss skripta med PowerShell (imperativt) och Python. Nu ska vi lära oss ett DEKLARATIVT sätt att hantera infrastruktur.

Problemet med manuell hantering:

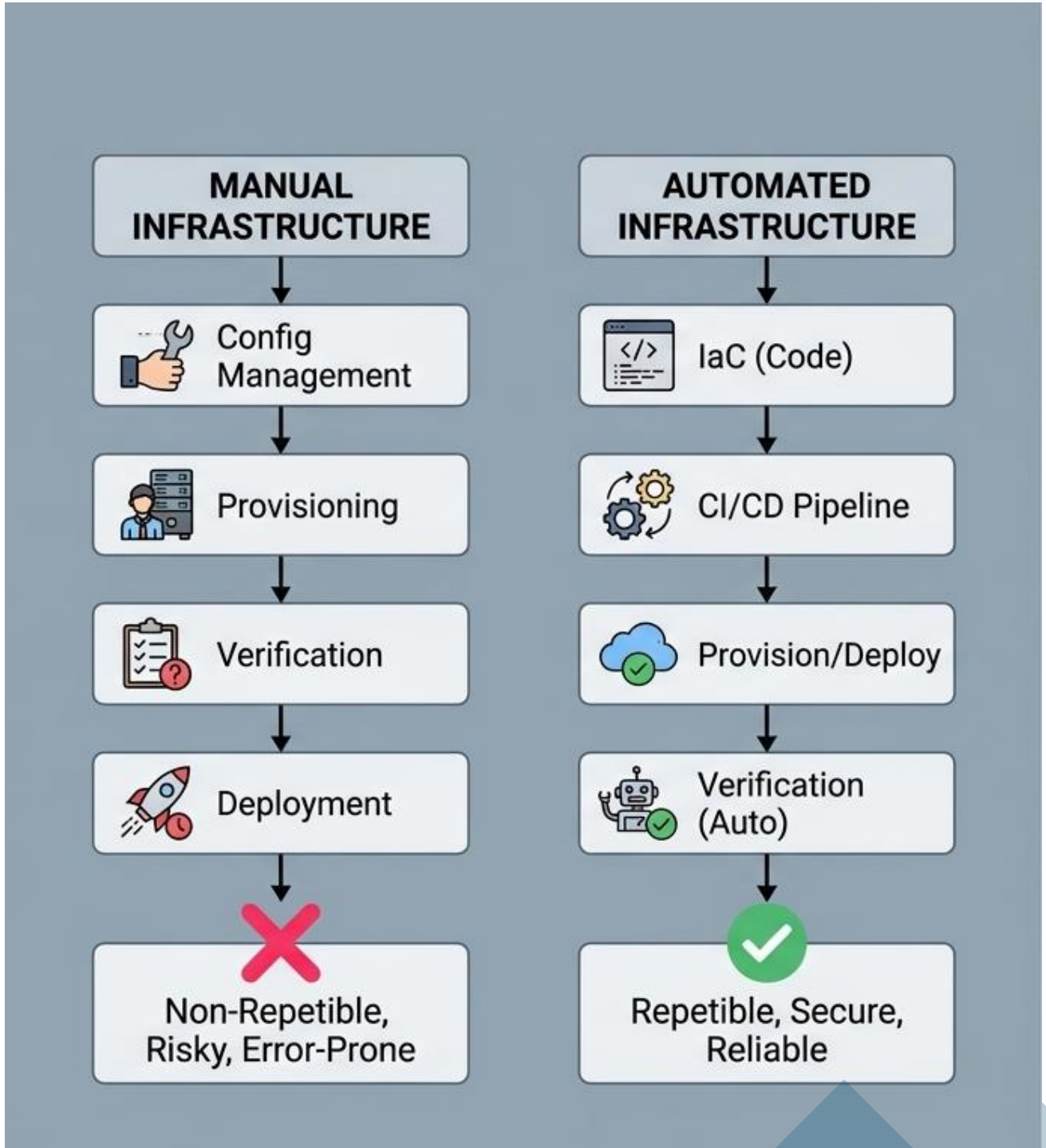
- ❌ Långsamt – timmar eller dagar för att konfigurera
- ❌ Felbenäget – mänskliga misstag
- ❌ Svårt att återskapa – "det funkar på min dator"
- ❌ Ingen historik – vem ändrade vad?

Manuellt vs Automatiserat



Med IaC

- ✅ **Versionshantering** – Allt sparas i Git, full historik och möjlighet att återställa
- ✅ **Konsekventa resultat** – Samma kod = samma infrastruktur, oavsett vem som kör den
- ✅ **Återanvändbart** – Skapa 100 servrar lika enkelt som 1
- ✅ **Snabbare driftsättning** – Från timmar till minuter
- **Enligt Microsoft**: IaC är en förutsättning för framgångsrik DevOps och betraktas som en hörnsten i modern molnhantering. I 2026 är IaC inte längre en nischad DevOps-praxis – det har blivit en kärnkompetens för organisationer som vill skala infrastruktur säkert.


MANUAL INFRASTRUCTURE
Config Management
Provisioning
Verification
Deployment
Non-Repetible, Risky, Error-Prone
AUTOMATED INFRASTRUCTURE
IaC (Code)
CI/CD Pipeline
Provision/Deploy
Verification (Auto)
Repetible, Secure, Reliable

Från Verkligheten – IaC i Svenska och Internationella Företag

IaC är standard i moderna IT-organisationer, både i Sverige och internationellt.

- **RBC (Royal Bank of Canada)** Byggde en centraliserad IaC-plattform för att hantera tusentals arbetsbelastningar över hybridmiljöer med GitOps och policy-as-code
- **TD Bank (Kanada)** Automatiserade nätverksuppdateringar i 1 300 filialer – 12 850 rader kod sparade 1 360 arbetstimmar
- **Volvo (Sverige)** Automatiserar driftsättning med Terraform och Ansible
- **Spotify (Sverige)** Provisionerar molninfrastruktur med Terraform
- **Klarna (Sverige)** Konfigurerar servrar med Ansible och Terraform

- **Varför detta är viktigt:** IaC används i finanssektorn, tech-sektorn och offentlig sektor för att säkerställa säkerhet, spårbarhet och snabb driftsättning. Även banker som TD Bank använder IaC – men alltid med mänsklig granskning av koden.

Imperativ vs Deklarativ – Två Sätt att Automatisera

- Vi har redan använt imperativa skript (PowerShell). Nu ska vi lära oss deklarativa – skillnaderna är viktiga!
- **Imperativ = Steg-för-steg-instruktioner**
- **Exempel: Baka en kaka**
- 1. Sätt ugnen på 175 grader
- 2. Smält smöret
- 3. Vispa socker och ägg
- 4. Blanda i mjölet

- **Deklarativ = Beskriv önskat resultat**
- Exempel: Baka en kaka
- "Jag vill ha en färdig kaka"

Imperativ (PowerShell)	Deklarativ (Terraform)
Beskriver HUR (steg-för-steg)	Beskriver VAD (önskat slutläge)
"Skapa mapp, kopiera fil, starta tjänst"	"Mappen SKA finnas, filen SKA finnas, tjänsten SKA vara igång"
Ordningen spelar roll	Ordningen spelar ingen roll – Terraform löser det
Exempel: PowerShell, Bash, Azure CLI	Exempel: Terraform, Ansible, ARM

I IT-världen

Enligt Microsoft: Deklarativt är **idempotent** – samma kod kan köras om och om igen utan negativa bieffekter. Detta är en av de största fördelarna med IaC-verktyg som Terraform.

Idempotensmotorn



Varför det är viktigt

Scenario	Utan Idempotens vs. Med Terraform
Autoskalning / CI-CD	**Utan:** Skript kraschar om resurser redan finns. **Med:** Fungerar alltid oavsett hur många gånger koden körs.
Avbruten deployment	**Utan:** Måste rensa upp dubbletter manuellt. **Med:** Kör bara koden igen – säkert.

Exempel – INTE idempotent:

powershell

```powershell
# Om användaren redan finns -> kraschar!
New-LocalUser -Name "Erik" -Password $password
```

Exempel – IDEMPOTENT (i Terraform):

hcl

```hcl
resource "local_file" "config" {
  content  = "Hej världen!"
  filename = "C:/temp/config.txt"
}
```

Enligt Microsoft: Idempotens är avgörande för autoskalning, återställning och kontinuerlig driftsättning. Alla IaC-verktyg är idempotenta – det är därför vi använder dem!

Varför idempotens är viktigt i praktiken

Scenario	Utan idempotens	Med idempotens (Terraform)
Autoskalning	Skript kraschar om resurser redan finns	Fungerar alltid
Avbruten deployment	Måste börja om – risk för dubbletter	Kör bara om igen – säkert!
CI/CD-pipeline	Kan inte köras flera gånger	Kan köras hur många gånger som helst
Disaster recovery	Svårt att återskapa	Kör koden – miljön återskapas

Terraform är det mest populära verktyget för Infrastructure as Code

Vad är Terraform?

Begrepp	Förklaring	Varför viktigt?
Provider	Talar om för Terraform vilken plattform som används (local, AWS, Azure)	Utan provider vet Terraform inte var den ska skapa resurser
Resource	En byggsten (fil, mapp, VM, nätverk)	Allt du skapar i Terraform är en resource
State	Terraforms "minne" – håller reda på vad som har skapats	Utan state vet Terraform inte vad som redan finns
HCL	HashiCorp Configuration Language – Terraforms språk	Språket du skriver infrastruktur i



Terraform arbetsflöde

Enligt Terraform-dokumentationen: `terraform plan` visar vad som kommer att ändras INNAN du kör `apply` – en viktig säkerhetsfunktion.



HCL – Språket i Terraform

- HCL (HashiCorp Configuration Language) är Terraforms deklarativa språk
- HCL är designat för att vara lätt att läsa för människor, men kraftfullt nog för att beskriva komplex infrastruktur. Det är deklarativt – du beskriver det önskade slutläget, och Terraform räknar ut hur du når dit.

Grundläggande syntax – en resource

hcl

```hcl
# Detta är en kommentar (fungerar med
// och #)


resource "local_file" "example" {
  # Argument - beskriver resursens
egenskaper
  filename = "C:/temp/hello.txt"
  content  = "Hej världen!"
}
```

Viktiga blocktyper



- **resource** Deklarerar en resurs som ska skapas `resource "local_file" "example"`
- **variable** Deklarerar en variabel (input) `variable "server_name"`
- **output** Visar information efter körning `output "file_path"`
- **data** Läser befintlig information

  `data "local_file" "config"`
- **provider** Anger vilken plattform som används `provider "local" { }`



HCL-regler att komma ihåg

- Namn ska vara gemener, siffror och understreck: `aws_vpc`, `local_file`
- `#` eller `//` för kommentarer
- Block har alltid `{ ... }` med argument inuti
- Filändelse: `.tf` (t.ex. `main.tf`, `variables.tf`)
- **Enligt KodeKloud:** HCL är inte ett programmeringsspråk – det saknar loopar och if-satser. Istället används `count` och `for_each` för att skapa flera resurser.

Providers och Resources – Grunden i IaC

Två begrepp som är nyckeln till att förstå Terraform:

Provider = "Vilken plattform?"

hcl

provider **"local" { }**
Detta talar om för Terraform att jobba på din dator

local Din lokala dator (filer, mappar) **provider "local" { }**

aws Amazon Web Services **provider "aws" { region = "eu-north-1" }**

azure Microsoft Azure **provider "azurerm" { features {} }**

Provider (Plattformen) - Local

```
provider "local" {
}
```

Provider = Vilken plattform?
Anger arbetsytan. Local providern jobbar direkt på din dator.

Perfekt för att rendera mallar eller lära sig IaC helt utan molnkostnader.

Resource (Byggstenen) - local_file

```
resource
  "local_file"
  "hello" {
  ...
}
```

Resource =
Vad ska skapas?

local_file: Skapar en vanlig textfil.

Resource (Byggstenen) - local_sensitive_file

```
resource
  "local_sensitive_file"
  "secret" {
  ...
}
```

Resource =
Vad ska skapas?

local_sensitive_file: Innehåll döljs i output för säkerhet.

Annoterat Blåkopia (Annotated Blueprint)

Kroppen
Block har alltid { ... } med argument inuti.

Kommentar
Fungerar med # eller //.

Blocktyp
Deklarerar att något ska skapas.

Resurstyp
Anger vilken specifik resurs inom providern. (Kräver alltid understreck).

Lokalt namn
Används bara inom koden för referens.

```
# Detta är en kommentar
resource "local_file" "example" {
  filename = "C:/temp/hello.txt"
  content  = "Hej världen!"
}
```

Argument
(egenskaperna)

hcl

```hcl
resource "local_file" "hello" {
  filename = "C:/temp/hello.txt"
  content  = "Hej världen från Terraform!"
}
```

local_file En textfil på din dator resource "local_file" "config"

local_sensitive_file En textfil med känsligt innehåll (dolt i output) resource "local_sensitive_file" "creds"

Resource = "Vad ska skapas?"

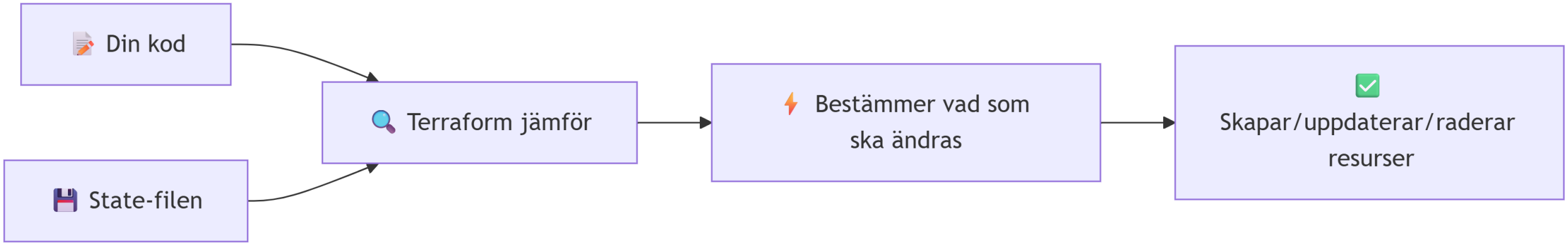
Enligt Spacelift: Local providern är idealisk för att skapa konfigurationsfiler, rendera mallar och dela data mellan moduler. Perfekt för att lära sig Terraform utan molnkostnader!

State – Terraforms "Minne"

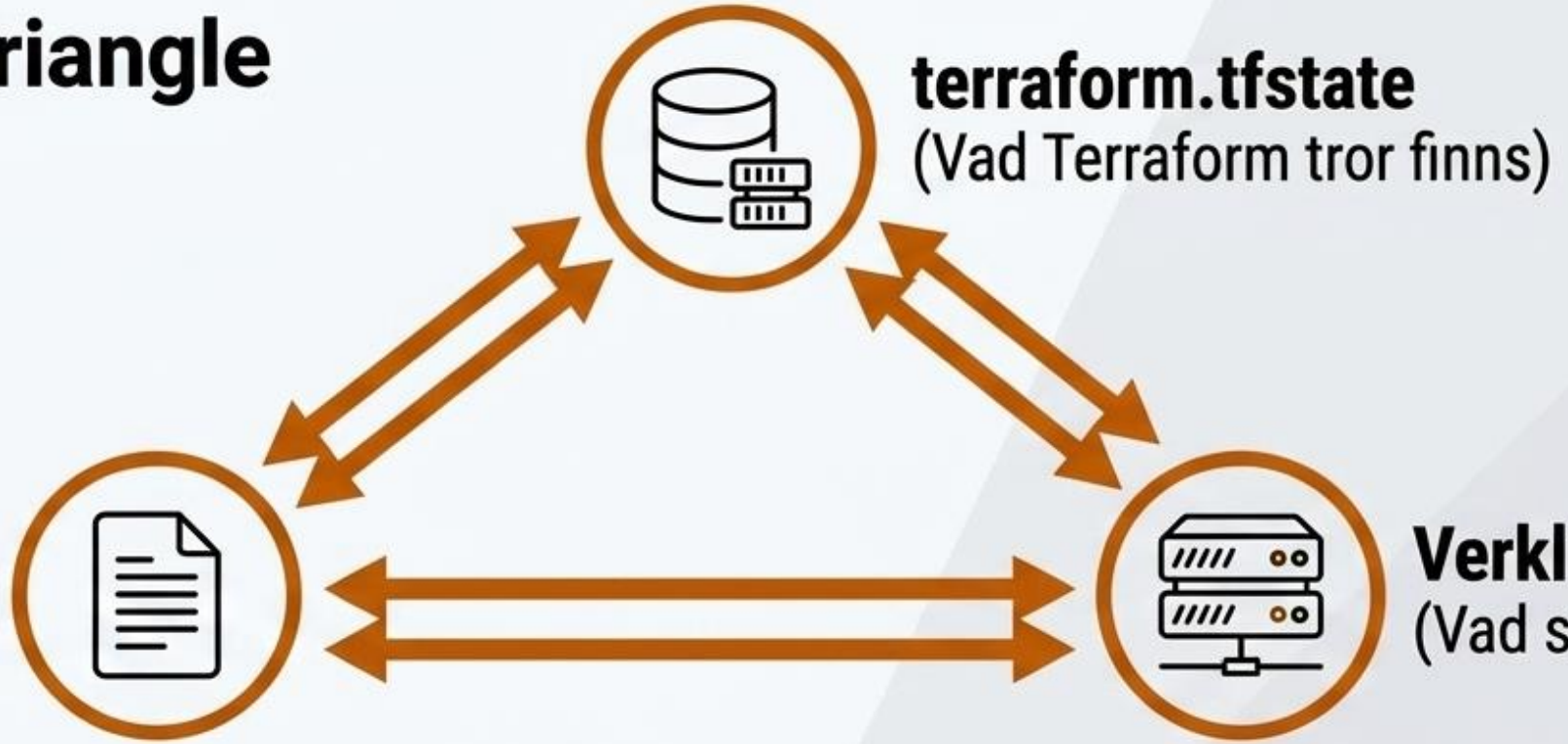
- State = Terraforms minne – håller reda på vad som har skapats.
- **Vilka resurser finns?** local_file.hello
- **Vilka ID:n har de?** Filens sökväg
- **Vilka attribut?** Filens innehåll

State finns i filen terraform.tfstate

Diagram – hur Terraform använder state:



The State Triangle



VARNING: Känslig Data!

State-filen lagrar allt i klartext (inkl. API-nycklar och lösenord).

Regel 1: Dela den ALDRIG offentligt. Lägg till i .gitignore.
Regel 2: Använd remote state (S3, Azure Storage) för team.
Insikt: Dela upp state per miljö. En enda stor state-fil är långsam och farlig vid misstag.

VARNING – VIKTIGT!

State-filen kan innehålla **känslig information** (API-nycklar, lösenord)

Dela den ALDRIG offentligt!

Lägg till terraform.tfstate i .gitignore!

Använd **remote state** (S3, Azure Storage) för teamarbete

Enligt Scalr: En stor state-fil är långsam och en enda miss har stor påverkan. Dela upp state per miljö och komponent – t.ex. separat state för nätverk, databaser och applikationer.



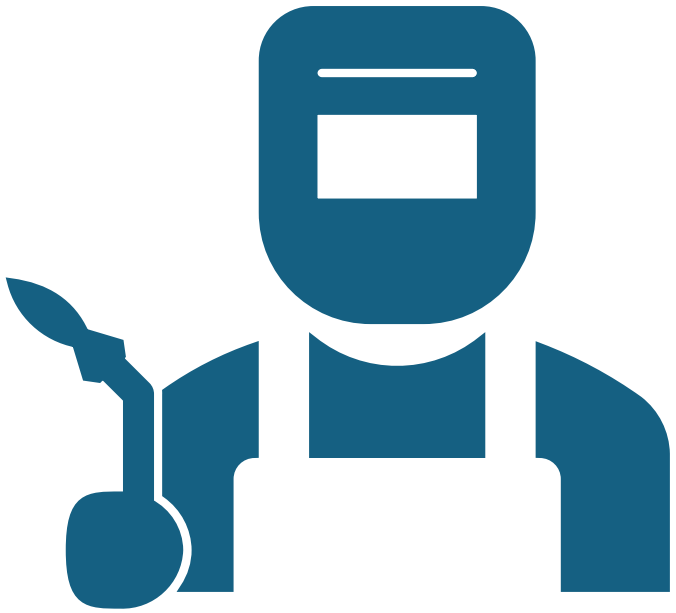
Local Provider – Din Första Terraform-konfiguration

Local providern är perfekt för att lära sig Terraform – inga molnkonton eller kostnader!



Koden – två filer

1. main.tf – beskriver vad som ska skapas



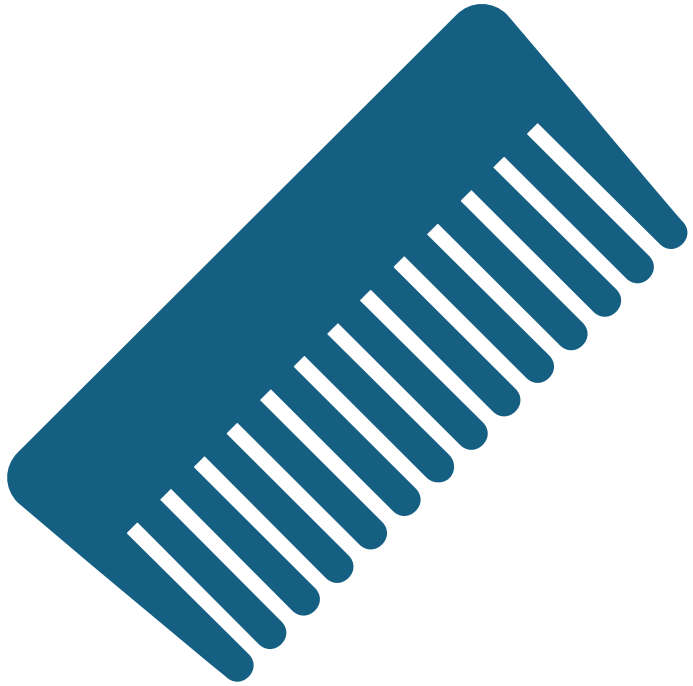
```hcl
# main.tf – Huvudkonfigurationen

# Provider – jobba på din dator
provider "local" { }

# Resource – skapa en fil
resource "local_file" "hello" {
  filename = "C:/temp/terraform_hello.txt"
  content  = "Hej från Terraform!"
}

# Resource – skapa en JSON-fil med jsonencode()
resource "local_file" "config" {
  filename = "C:/temp/config.json"
  content  = jsonencode({
    server = "WEB01"
    port   = 80
    ssl    = true
```

2. variables.tf – deklarera variabler

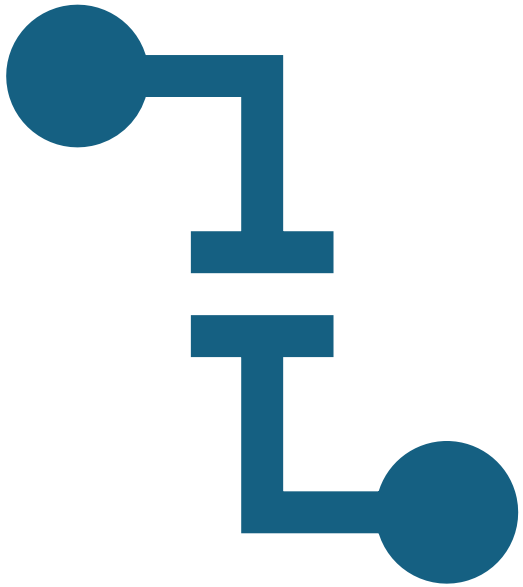


```
# variables.tf - Inmatningsvariabler

variable "environment" {
  type        = string
  description = "Miljö (dev, test, prod)"
  default     = "development"
  sensitive   = false  # Inte känslig data
}

variable "api_key" {
  type        = string
  description = "API-nyckel för extern
```

Så här kör du



- **terraform init** Laddar ner provider-plugins Alltid först!
- **terraform validate** Kontrollerar syntax och semantik Fångar fel innan plan
- **terraform plan** Visar vad som kommer att ändras Alltid innan apply!
- **terraform apply -auto-approve** Genomför ändringarna Utan -auto-approve frågar den om bekräftelse
- **Enligt HashiCorp**: terraform validate fångar syntaxfel och missade referenser INNAN du kör plan eller apply. Använd terraform fmt för att automatiskt formatera koden enligt standard.

Variabler och Outputs – Återanvändbar IaC

- Variabler = Gör din kod flexibel och återanvändbar för olika miljöer
- Bästa praxis för variabler:
- Ange alltid **type** - Förhindrar felaktiga värden
- Lägg till **description** - Gör koden självdokumenterande
- Ange **default** för valfria variabler - Enklare att använda
- Använd **sensitive = true** för hemligheter - Döljs i loggar och output
- Använd **validation** för att begränsa värden - Förhindrar felaktiga inställningar



Exempel – validering av variabel

```
variable "environment" {
  type        = string
  description = "Miljö: dev, test, staging, production"

  validation {
    condition     = contains(["dev", "test", "staging", "production"], var.environment)
    error_message = "Environment måste vara dev, test, staging eller production."
  }
}

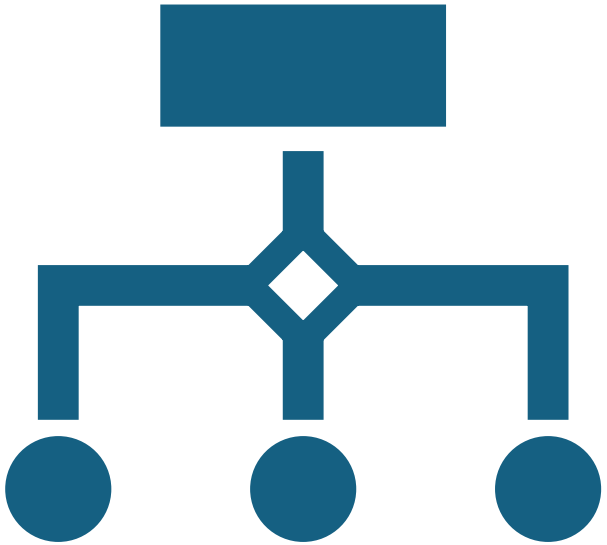
variable "instance_count" {
  type        = number
  description = "Antal servrar att skapa"

  validation {
    condition     = var.instance_count > 0 && var.instance_count <= 10
    error_message = "Instance count måste vara mellan 1 och 10."
  }
}
```

Outputs – visa resultat

```hcl
output "file_path" {
  description = "Sökväg till skapad fil"
  value       = local_file.example.filename
}

output "sensitive_credential" {
  description = "API-nyckel"
  value       = var.api_key
  sensitive   = true   # Döljs i loggar
}
```



variables.tf (Input)

```
variable "environment" {
  type        = string
  description = "Miljö"
  default     = "dev"

  validation {
    condition     = contains(["dev",
"test", "prod"], var.environment)
    error_message = "Måste vara
dev, test, eller prod."
  }
}
```

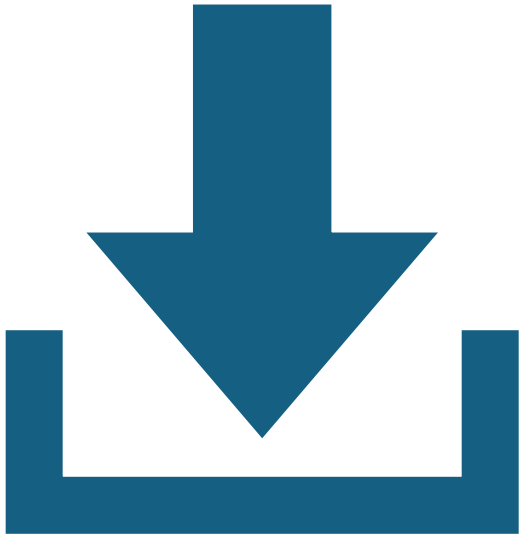
- ✓ Ange alltid type
- ✓ Använd description
- ✓ Skydda hemligheter med sensitive = true

Terminal Output

```
output "sensitive_credential" {
  value     = var.api_key
  sensitive = true
}
```

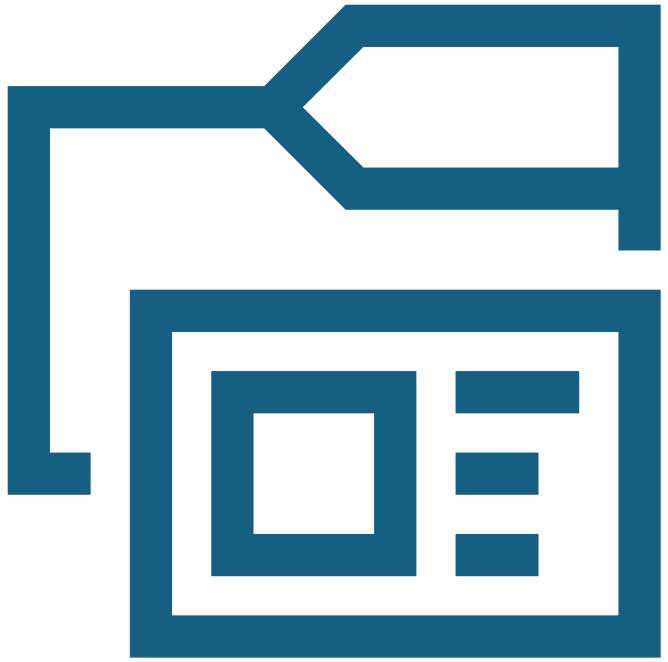
Outputs kan döljas i loggar. Hämta dem programmatiskt när de behövs med: `terraform output -raw`

- **Outputs med sensitive:** Enligt Terraform-dokumentationen kan du hämta känsliga outputs med `terraform output -json` eller `terraform output -raw` för att få ut värdena när du verkligen behöver dem (t.ex. i skript).
- **Enligt Scalr:** Använd `terraform.tfvars`-filer för att hålla miljöspecifika värden (dev.tfvars, prod.tfvars) separata från koden. Lägg ALDRIG in hemligheter i vanliga `.tfvars`-filer som du committar.



Data Sources – Läsa Befintlig Information

- Data sources låter Terraform läsa information från befintliga resurser – utan att skapa något nytt.
- Data sources = Read-only queries – de hämtar information, men skapar ingenting.





Din Kod



Miljön

Resource (Write - Skapa / Ändra)

```
resource "local_file" {
```

Resultat: En ny fil skapas eller modifieras.

Data Source (Read-Only - Hämta Information)

Resultat: Läser in existerande JSON/YAML utan att modifiera något.

```
data "local_file" "existing"
```

Data sources är riskfria sökfrågor. De möjliggör dynamisk kod (t.ex. att hämta senaste AMI-ID:t) utan att riskera oavsiktliga ändringar.

Exempel – läsa en befintlig fil:

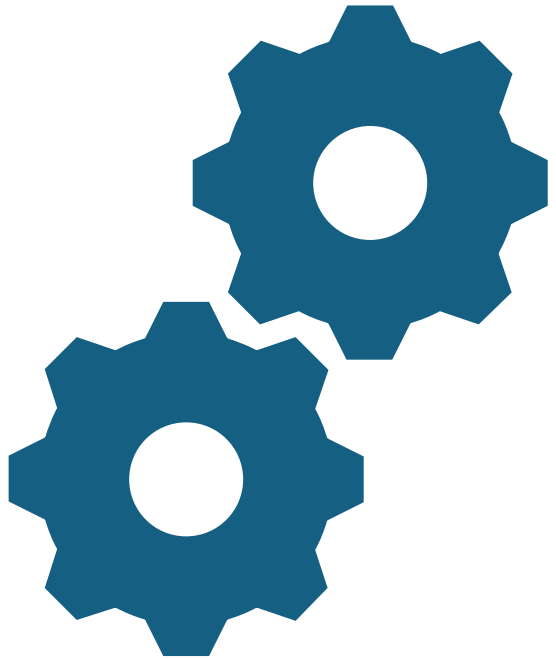
```hcl
# data.tf - Läs en befintlig fil

data "local_file" "existing" {
  filename = "C:/temp/existing_config.json"
}

output "content" {
  value = data.local_file.existing.content  # Innehållet i filen
}
```



Exempel – läsa innehåll från fil och använda:



```hcl
# main.tf - Använd data från fil

data "local_file" "policy_json" {
  filename = "${path.module}/policy.json"
}

resource "local_file" "deployment_ready" {
  filename = "C:/temp/deployment_ready.txt"
  content  = data.local_file.policy_json.content  # Använd innehållet
```

När använder man data sources?

- **Läsa konfigurationsfiler** JSON/YAML-filer som finns i repositoryt
- **Hämta information från molnet** T.ex. senaste AMI-ID i AWS
- **Undvika hårdkodning** Läs värden från fil istället för att skriva in dem i koden
- Enligt HashiCorp: Data sources är ett säkert sätt att läsa befintlig information – de skapar inga nya resurser, bara läser. När filen på disk ändras kommer nästa terraform plan att visa ändringen, eftersom data source läser från filen igen.





Workshop – Din Första Terraform-konfiguration

Mål: Skapa din första Terraform-konfiguration och se IaC i praktiken!

Steg 1 – Skapa projektmapp och filer

Skapa en mapp C:/IaC/project/ och där skapar du två filer

main.tf

```hcl
# main.tf – Huvudkonfiguration
provider "local" { }

resource "local_file" "hello" {
  filename = "C:/temp/terraform_hello.txt"
  content  = "Hej från Terraform!"
}

output "file_created" {
  value = local_file.hello.filename
}
```

variables.tf

```hcl
# variables.tf - Variabler
variable "environment" {
  type    = string
  default = "development"
}
```

Steg 2 – Kör Terraform

```bash
# Navigera till mappen
cd C:/IaC/project/

# Initiera – ladda ner providers
terraform init

# Validera syntax (fångar missar som saknade parenteser!)
terraform validate

# Planera (se vad som ska hända)
terraform plan

# Applicera
terraform apply -auto-approve
```

Steg 3 – Kontrollera att filen skapades

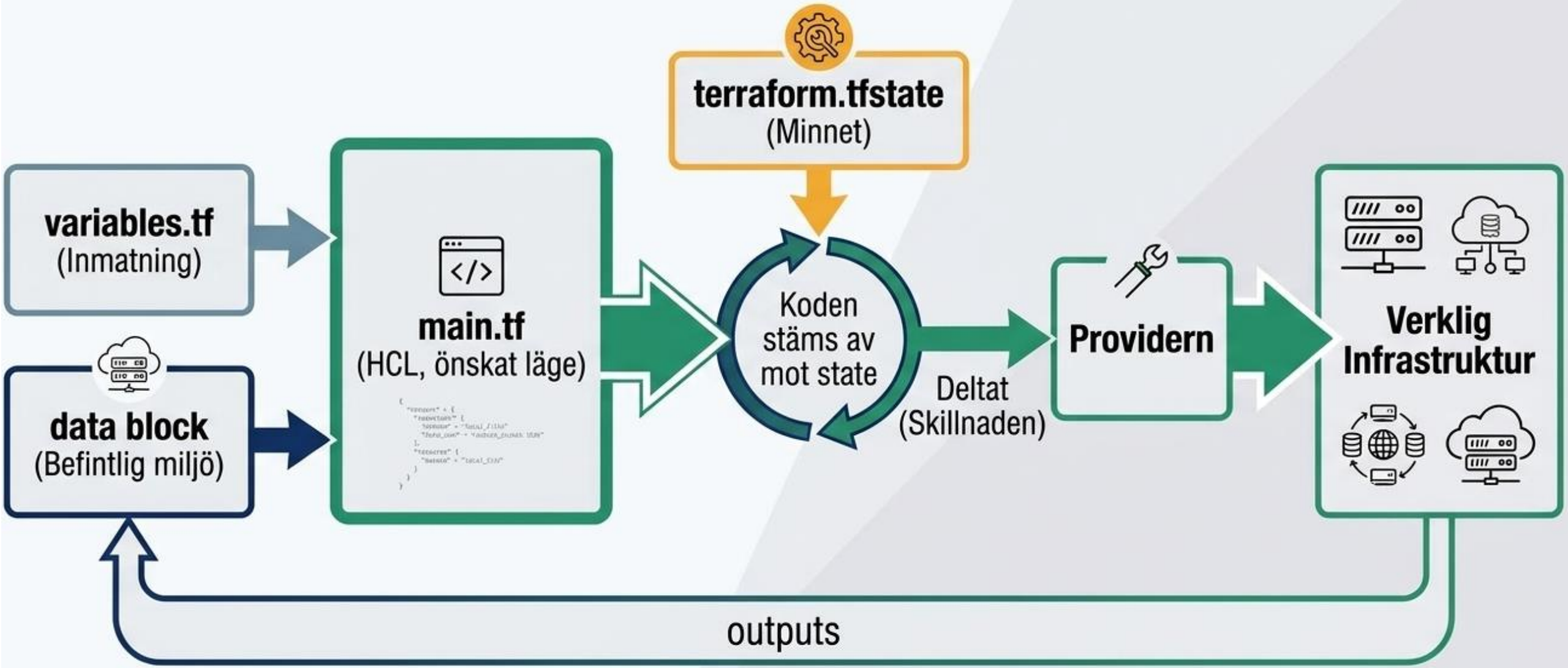
- `bash`
- `Get-Content C:/temp/terraform_hello.txt`

Extra utmaning

- Lägg till en till resource som skapar en JSON-fil med `jsonencode()`
- Använd en variabel för att styra filnamnet
- **Diskussionsfråga:** Varför är det bra att Terraform kan jämföra state med verkligheten innan den gör ändringar? Varför är detta bättre än att bara köra ett skript?
- **Proffstips:** Använd `terraform validate` ofta – det fångar syntaxfel och missade referenser INNAN du kör `plan` eller `apply`.

Sammanfogning av Terraform-motorn

Hur VAD-beskrivningen blir till verklighet.




1. INIT
Ladda ner providers.
(Körs alltid först i ett nytt projekt).
2. VALIDATE
Syntaxkontroll.
Fångar saknade parenteser direkt.
3. PLAN
Jämför koden med state.
Visar exakt vad som kommer att skapas, ändras eller raderas INNAN det händer. Detta är IaC:s största fördel mot skript.
4. APPLY
Genomför ändringarna.
Kräver bekräftelse.

Din Kod

Filer att skapa



C:/IaC/project/

├─ main.tf

└─ variables.tf

```
1  provider "local" { }
2  resource "local_file" "hello" { ... }
```

```
1  variable "environment" { ... }
```

Din Terminal

Kommandon att köra

```
> terraform init
> terraform validate
> terraform plan
> terraform apply
> Get-Content C:/temp/terraform_hello.txt
```

Inga manuella klick. 100% repeterbart. Välkommen till Infrastructure as Code.

Sammanfattning – Vad Har Vi Lärt Oss Idag?

- **Infrastructure as Code (IaC):**
- Beskriv infrastruktur med kod istället för manuella processer
- Allt sparas i Git – full historik och återställningsmöjlighet
- **Imperativ vs Deklarativ:**
- Imperativ = beskriv HUR (PowerShell, Bash)
- Deklarativ = beskriv VAD (Terraform, Ansible)
- **Idempotens:**
- Samma kod kan köras om och om igen utan negativa bieffekter
- Avgörande för autoskalning, återställning och CI/CD
- **Terraform:**
- Provider = plattform (local, AWS, Azure)
- Resource = vad som ska skapas (fil, mapp, VM)
- State = Terraforms minne (vad har skapats?)
- HCL = Terraforms deklarativa språk
- **Terraform arbetsflöde**: init → validate → plan → apply → (destroy)
- **Data sources**: Läs befintlig information, skapa inga nya resurser

Quiz:

https://forms.gle/qBeWT2iE216jzFEL6